

Análisis y Diseño

Lista de Tareas — DemoCopilot

Junio 2026

1. Objetivo del proyecto

Aplicación web CRUD de gestión de tareas personales construida como demo didáctica para el curso de GitHub Copilot. Permite crear, consultar, actualizar y eliminar tareas, con soporte de **plantillas reutilizables** y **tareas repetitivas** con generación automática de la siguiente ocurrencia al completar. El objetivo prioritario es la claridad del código sobre la sofisticación arquitectónica.

2. Stack tecnológico

Tecnología	Versión	Rol
ASP.NET Core	8	API REST / backend (Controllers)
Entity Framework Core	8	ORM / acceso a datos
SQLite	—	Base de datos embebida, sin infraestructura externa
xUnit + Moq	—	Tests unitarios

3. Arquitectura de capas

Estructura de capas plana y legible en pantalla, sin patrones complejos (sin CQRS ni Mediator).

Capa	Carpeta	Responsabilidad
Modelos de dominio	Models/	Entidades del negocio (TodoItem, PlantillaTarea, TipoRecurrencia)
Acceso a datos	Data/	DbContext y configuración de EF Core
Lógica de negocio	LogicaNegocio/	IXxxLogica + XxxLogica — reglas de negocio y acceso a datos
Servicios	Services/	IXxxService + XxxService — orquestación y mapeo DTO ↔ entidad
API / Controladores	Controllers/	Orquestación HTTP, sin lógica de negocio
Tests	Tests/	Proyecto xUnit separado

Flujo de llamadas

Controller → [DTO] → Service → [Entidad] → LogicaNegocio → DbContext

Reglas de diseño

- Inyección de dependencias por constructor, nunca *new* directo de servicios.
- *async/await* en todos los métodos que accedan a base de datos.
- Los controladores solo orquestan — sin lógica de negocio dentro de ellos.
- Prefijo **I** para interfaces: *ITodoService*, *IPlantillaService*, etc.

4. Modelo de datos

TodoItem

Campo	Tipo	Descripción
Id	int	Clave primaria, autogenerada
Title	string	Título o descripción de la tarea. Requerido
IsCompleted	bool	Indica si la tarea está completada. Por defecto false
CreatedAt	DateTime	Fecha y hora de creación, asignada al crear
EsRepetitiva	bool	Indica si la tarea se repite automáticamente al completarse
Recurrencia	TipoRecurrencia?	Periodicidad: Diaria, Semanal o Mensual
ProximaFecha	DateTime?	Fecha calculada para la siguiente ocurrencia
PlantillaId	int?	FK opcional a PlantillaTarea
UsuarioAsignadoId	int?	FK opcional a UsuarioAsignado

```
public class TodoItem
{
    public int Id { get; set; }
    public string Title { get; set; } = string.Empty;
    public bool IsCompleted { get; set; }
    public DateTime CreatedAt { get; set; }
    public bool EsRepetitiva { get; set; }
    public TipoRecurrencia? Recurrencia { get; set; }
    public DateTime? ProximaFecha { get; set; }
    public int? PlantillaId { get; set; }
    public PlantillaTarea? Plantilla { get; set; }
    public int? UsuarioAsignadoId { get; set; }
    public UsuarioAsignado? UsuarioAsignado { get; set; }
}
```

UsuarioAsignado

Usuario que puede ser asignado a una o varias tareas. La FK en TodoItem es opcional (SetNull al eliminar el usuario).

Campo	Tipo	Descripción
Id	int	Clave primaria, autogenerada
Nombre	string	Nombre completo. Requerido, máx. 100 caracteres
Email	string	Correo electrónico. Requerido, único, máx. 200 caracteres

PlantillaTarea

Plantilla reutilizable desde la que se pueden generar nuevas tareas con valores predefinidos.

Campo	Tipo	Descripción
Id	int	Clave primaria, autogenerada
Título	string	Título por defecto de la tarea generada. Requerido

Campo	Tipo	Descripción
EsRepetitiva	bool	Si las tareas generadas serán repetitivas
Recurrencia	TipoRecurrencia?	Periodicidad por defecto para las tareas generadas

TipoRecurrencia (enum)

```
public enum TipoRecurrencia
{
    Diaria,
    Semanal,
    Mensual
}
```

5. Endpoints API REST

Tareas — /api/tareas

Verbo	Ruta	Descripción	OK	Error
GET	/api/tareas	Listar todas las tareas	200	—
GET	/api/tareas/{id}	Obtener tarea por ID	200	404
POST	/api/tareas	Crear nueva tarea	201	400
PUT	/api/tareas/{id}	Actualizar título o estado	200	404/400
DELETE	/api/tareas/{id}	Eliminar tarea	204	404
POST	/api/tareas/{id}/completar	Completar; si es repetitiva genera siguiente ocurrencia	200	404

Usuarios asignados — /api/usuariosasignados

Verbo	Ruta	Descripción	OK	Error
GET	/api/usuariosasignados	Listar todos los usuarios	200	—
GET	/api/usuariosasignados/{id}	Obtener usuario por ID	200	404
POST	/api/usuariosasignados	Crear usuario nuevo	201	400
PUT	/api/usuariosasignados/{id}	Actualizar nombre o email	200	404/400
DELETE	/api/usuariosasignados/{id}	Eliminar (tareas quedan sin usuario asignado)	204	404

Plantillas — /api/plantillas

Verbo	Ruta	Descripción	OK	Error
GET	/api/plantillas	Listar todas las plantillas	200	—
GET	/api/plantillas/{id}	Obtener plantilla por ID	200	404

Verbo	Ruta	Descripción	OK	Error
POST	/api/plantillas	Crear nueva plantilla	201	400
PUT	/api/plantillas/{id}	Actualizar plantilla	200	404/400
DELETE	/api/plantillas/{id}	Eliminar plantilla	204	404
POST	/api/plantillas/{id}/instanciar	Crear tarea a partir de la plantilla	201	404

6. Datos de ejemplo

El seeder se invoca desde Program.cs al arrancar la aplicación y solo inserta datos si la tabla está vacía.

Entidad	Registros de ejemplo
UsuarioAsignado	Ana García (ana@demo.com), Carlos López (carlos@demo.com)
PlantillaTarea	"Revisión semanal" (semanal), "Backup diario" (diaria)
TodoItem	"Configurar entorno" (completada), "Revisar PR pendientes" (repetitiva semanal, asignada a Ana)

7. Decisiones de diseño

- **SQLite sobre SQL Server:** Base de datos embebida, sin instalación ni configuración de servidor. Ideal para demo local y didáctica.
- **Controllers sobre Minimal API:** Los controladores con atributos son más explícitos y fáciles de leer en pantalla durante una presentación.
- **Inyección de dependencias:** Patrón estándar de .NET; desacopla la lógica de negocio y facilita los tests con mocks.
- **Interfaz IToDoService:** Permite sustituir la implementación por un mock en los tests sin tocar el controlador.
- **Recurrencia calculada en el servicio:** La lógica de generar la siguiente ocurrencia vive en TodoService.Completar(), no en el controlador ni en el modelo.
- **POST /completar en lugar de PUT:** La acción tiene un efecto secundario (crear nueva ocurrencia), por lo que merece su propio endpoint de acción.
- **Plantilla como entidad independiente:** PlantillaTarea es una entidad propia para permitir gestionar y reutilizar plantillas sin acoplarlas a las tareas.
- **Sin paginación ni autenticación:** Fuera del alcance de la demo; añadir solo si se solicita explícitamente.
- **Código en castellano:** Nombres de clases, métodos y variables en español para coherencia con el público hispanohablante del curso.

8. Pendientes / Preguntas abiertas

- **Frontend:** ¿Razor Pages o React + Vite? Pendiente de decisión en el curso.
- **Paginación:** No incluida. Si el volumen crece, se añadiría skip/take al endpoint GET.
- **Autenticación:** No incluida. Posible extensión futura con JWT o cookies.
- **Filtros y búsqueda:** Posible extensión: GET /api/tareas?completada=true.
- **Soft delete vs hard delete:** Actualmente se borra físicamente. Podría añadirse campo DeletedAt si se requiere historial.
- **Job de recurrencia automática:** La siguiente ocurrencia se genera solo al llamar a POST /completar. Para generación automática habría que añadir un BackgroundService.
- **Recurrencia personalizada:** El enum cubre Diaria/Semanal/Mensual. Para patrones más complejos habría que extender el modelo.
- **Herencia plantilla → tareas:** Si se modifica una plantilla, las tareas ya creadas no se actualizan. Comportamiento a definir si se requiere propagación.